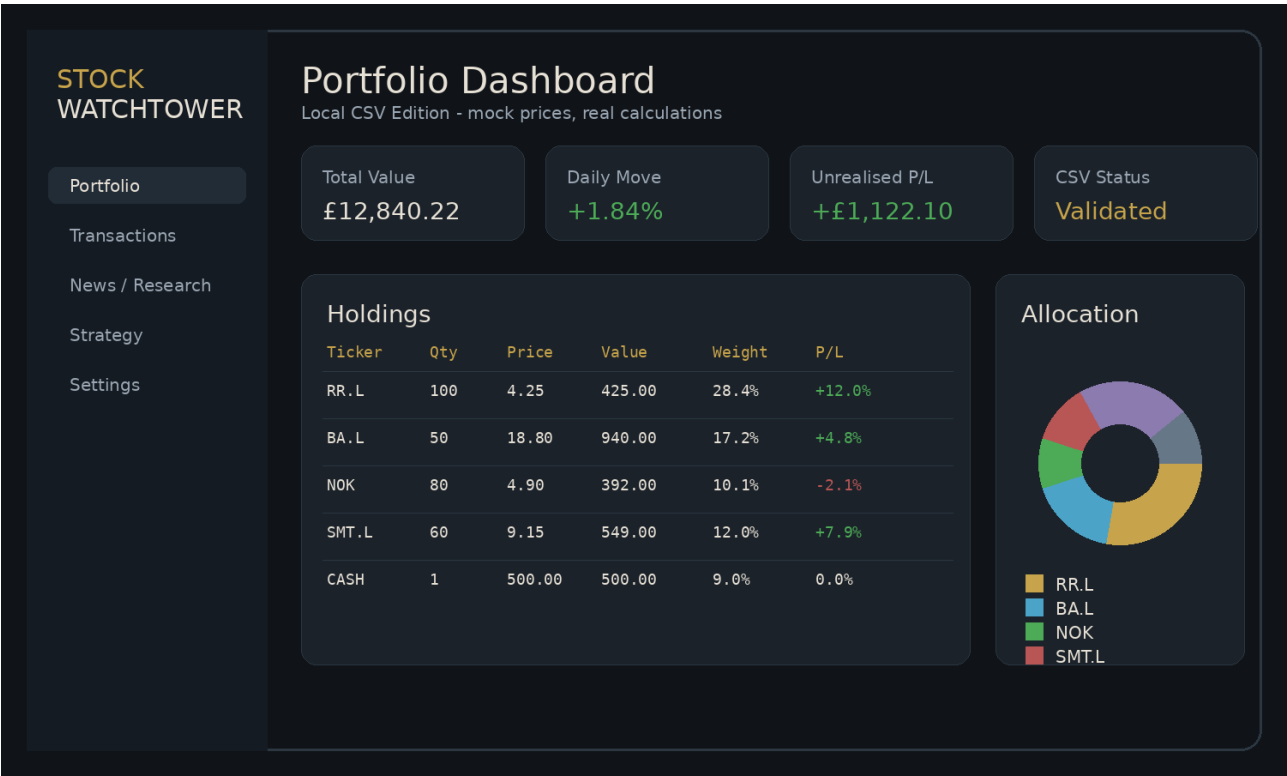


Stock Watchtower

Local CSV Edition

A scaffold-first PDF guide for building a personal Python portfolio dashboard with Streamlit, CSV files, desktop launchers, and a design identity.



Prepared for a VSCode + GitHub + ChatGPT-assisted workflow. The first objective is not perfection; it is to create a clean set of blank canvases, then make one small section work at a time.

1. What we are building

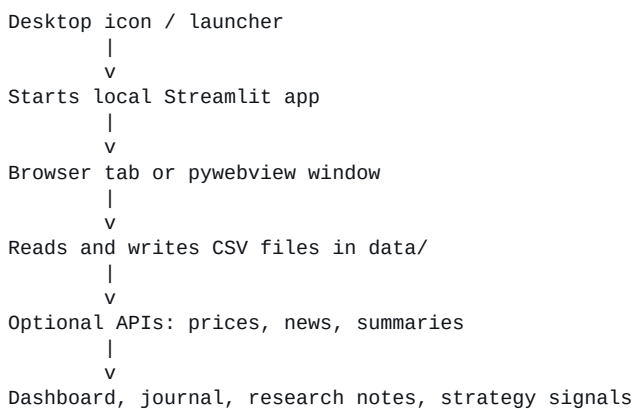
The aim is a personal, local-first portfolio dashboard. It runs on your machine, reads and writes plain CSV files, and opens through a local browser or desktop-style wrapper. The app does not need hosting, accounts, or a cloud database at the start.

The first build is deliberately small: holdings.csv is the source of truth, mock prices power the dashboard, and the app calculates portfolio value, gain/loss, and allocation. After that, we add in-app editing, backups, transactions, a desktop launcher, design polish, quote APIs, news/research, and strategy signals.

Decision	Chosen approach	Why
App framework	Streamlit	Fast local Python dashboard building with minimal frontend work.
Persistence	CSV files	Readable, editable in spreadsheet apps or raw text, easy to back up.
Database later?	Only if CSV becomes annoying	SQLite/Postgres can wait until the app needs stronger structure.
Hosting	None at first	Local use avoids deployment and account complexity.
Desktop feel	Launcher first, pywebview later	Start simple, then wrap the local web app when useful.

Technical grounding: Streamlit apps are normal Python scripts that are run with commands such as `streamlit run app.py` or `python -m streamlit run app.py`. Streamlit also supports theming through configuration options, commonly in `.streamlit/config.toml`. `pywebview` can later create a native desktop window that displays a local web app. See the reference section for official docs.

2. Target architecture



The Streamlit process is effectively your local backend and frontend together. That is what keeps the first version light. It still has proper internal modules - `calculations.py`, `csv_store.py`, `validation.py`, `quotes.py`, `news.py` - so that the app does not become one giant script.

3. Tools checklist

Tool	Purpose	Install/check
VSCode	Editor and project workspace	Open the stock-watchtower folder as a workspace.
Python 3.11+	Runs Streamlit and app logic	<code>python --version</code>
venv	Project-local Python environment	<code>python -m venv .venv</code>
Git	Version control	<code>git --version</code>
GitHub	Code backup and history	Private repo recommended if notes/data are included.
Streamlit	Local web app dashboard	<code>pip install streamlit</code>
pandas	CSV and table handling	<code>pip install pandas</code>
Plotly	Charts	<code>pip install plotly</code>

Use the virtual environment even for a personal app. It keeps package versions isolated from the rest of your machine. Python venv creates a lightweight environment with its own installed packages, separated from the base Python installation.

4. Your blank-canvas workflow

You like to manually create the folder structure and blank files first. That is a strong workflow for this project because it lets you see the app before it exists. Treat the folder tree as the map: every blank file is a labelled workbench.

VSCode scaffold view - blank canvases first

```

stock-watchtower/
  app.py
  launcher.py
  requirements.txt
  README.md
  .env.example
  .gitignore
  .streamlit/
    config.toml
  assets/
    logo-mark.png  icon.ico  theme_reference.md
  data/
    holdings.csv  transactions.csv  strategy_prof.
  cache/
    quotes_cache.csv  news_cache.csv
  src/
    calculations.py  csv_store.py  validation.py
    quotes.py  news.py  strategy.py  theme.py
  pages/
    1_Portfolio.py  2_Transactions.py  3_News_Research.py
  docs/
    PROJECT_CONTEXT.md  TASKS.md  BRAND.md  PROMPTS.md

```

Why this structure?

- data/ is your human-editable DB
- src/ holds reusable Python logic
- pages/ becomes the Streamlit navigation
- assets/ stores logo and app icon work
- docs/ keeps ChatGPT grounded

Rule of the build

Create the blank files first. Then fill one small milestone at a time. Your folders are the map; your prompts are the walking route.

Blank file naming style

For Python, prefer clear module names rather than Empty_Scripts.py. A blank script should tell you what it will become. For example:

Good:

```

src/calculations.py
src/csv_store.py
src/validation.py
src/quotes.py
src/news.py
src/strategy.py

```

Avoid:

```

Empty_Script_1.py
Misc_Functions.py
Final_Final_Utils.py

```

For this build, there is no need for blank-pages.html at the start because Streamlit creates the web interface from Python. If you still want HTML placeholders for future design thinking, put them in design/ or prototypes/, not in the active app path. That way they are reference canvases, not confusing runtime files.

Optional design-only HTML placeholders:

```

prototypes/
  dashboard_mock.html
  holding_card_mock.html

```

```
launcher_page_mock.html
```

```
Active Streamlit pages:
pages/
  1_Portfolio.py
  2_Transactions.py
  3_News_Research.py
  4_Strategy.py
  5_Settings.py
```

5. Create the project folder

macOS / Linux terminal

```
mkdir stock-watchtower
cd stock-watchtower

mkdir -p assets data cache backups/holdings exports src pages docs tests .streamlit

touch app.py launcher.py requirements.txt README.md .env.example .gitignore

touch .streamlit/config.toml

touch assets/theme_reference.md

touch data/holdings.csv data/transactions.csv data/strategy_profile.csv data/research_notes.csv

touch cache/quotes_cache.csv cache/news_cache.csv

touch src/__init__.py src/calculations.py src/csv_store.py src/mock_data.py src/validation.py

touch src/theme.py src/quotes.py src/news.py src/strategy.py src/charts.py

touch pages/1_Portfolio.py pages/2_Transactions.py pages/3_News_Research.py pages/4_Strategy.py
pages/5_Settings.py

touch docs/PROJECT_CONTEXT.md docs/TASKS.md docs/DECISIONS.md docs/PROMPTS.md docs/BRAND.md

touch tests/test_calculations.py tests/test_csv_store.py
```

Windows PowerShell

```
mkdir stock-watchtower
cd stock-watchtower

mkdir assets,data,cache,exports,src,pages,docs,tests,.streamlit
mkdir backups
mkdir backups\holdings

ni app.py,launcher.py,requirements.txt,README.md,.env.example,.gitignore -ItemType File
ni .streamlit\config.toml -ItemType File
ni assets\theme_reference.md -ItemType File
ni data\holdings.csv,data\transactions.csv,data\strategy_profile.csv,data\research_notes.csv -
ItemType File
ni cache\quotes_cache.csv,cache\news_cache.csv -ItemType File
ni src\__init__.py,src\calculations.py,src\csv_store.py,src\mock_data.py,src\validation.py -
ItemType File
ni src\theme.py,src\quotes.py,src\news.py,src\strategy.py,src\charts.py -ItemType File
ni pages\1_Portfolio.py,pages\2_Transactions.py,pages\3_News_Research.py,pages\4_Strategy.py,
pages\5_Settings.py -ItemType File
ni docs\PROJECT_CONTEXT.md,docs\TASKS.md,docs\DECISIONS.md,docs\PROMPTS.md,docs\BRAND.md -
ItemType File
ni tests\test_calculations.py,tests\test_csv_store.py -ItemType File
```

6. Create and activate the Python environment

macOS / Linux

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install streamlit pandas plotly pydantic python-dotenv pytest ruff
```

```
pip freeze > requirements.txt
```

Windows PowerShell

```
python -m venv .venv  
.\.venv\Scripts\Activate.ps1  
python -m pip install --upgrade pip  
pip install streamlit pandas plotly pydantic python-dotenv pytest ruff  
pip freeze > requirements.txt
```

If PowerShell blocks activation, run PowerShell as your normal user and use: Set-ExecutionPolicy -Scope CurrentUser RemoteSigned. Then try the activation command again. Do not loosen execution policy globally unless you understand the impact.

7. File purpose map

Path	Purpose
app.py	Main Streamlit entry point. Keep this small: page config, sidebar, top-level dashboard call.
src/calculations.py	Pure maths: current value, gain/loss, gain/loss percent, allocation weight. Easy to test.
src/csv_store.py	Read/write CSV files, create backups, safe file replacement.
src/validation.py	Check required columns, numeric fields, duplicate IDs, missing tickers.
src/mock_data.py	Sample holdings and mock prices for the first milestone.
src/theme.py	Brand colours and reusable UI constants.
src/quotes.py	Later: real quote provider, cache, stale data handling.
src/news.py	Later: ticker news/RSS retrieval and news cache.
src/strategy.py	Later: rule-based alerts and strategy profile logic.
pages/	Streamlit multipage sections. Fill these after the main app works.
data/	Human-editable CSV source files. Do not commit real personal financial data to a public repo.
cache/	Temporary quote/news cache files. Safe to delete/regenerate.
backups/	Timestamped backups created before the app writes CSV changes.
assets/	Logo, icon, theme reference, future screenshots.
docs/	Project memory for you and ChatGPT.

8. Data files: CSV as your database

CSV works here because the app is personal. You can edit holdings in a spreadsheet, raw text editor, or the app itself. The guardrail is validation: the app should check the file before trusting it.

data/holdings.csv starter

```
holding_id,name,ticker,exchange,currency,asset_type,quantity,average_cost,isin,figi,tags,notes,
created_at,updated_at
h001,Rolls-Royce,RR.L,LSE,GBP,stock,100,4.25,GB00B63H8491,,defence;uk;growth,Core UK defence
and SMR watch,2026-07-07,2026-07-07
h002,Nokia ADR,NOK,NYSE,USD,stock,50,4.80,US6549022043,,telecom;adr,US-listed Nokia ADR,2026-07-
07,2026-07-07
h003,Cash GBP,CASH,LOCAL,GBP,cash,1,500.00,,,cash,Dry powder,2026-07-07,2026-07-07
```

data/transactions.csv starter

```
transaction_id,date,holding_id,type,quantity,price,fees,currency,notes
t001,2026-07-07,h001,buy,100,4.25,1.00,GBP,Initial position
```

data/strategy_profile.csv starter

```
key,value
horizon,long
risk_tolerance,medium_high
max_single_holding_percent,25
dip_watch_threshold_percent,4
preferred_sectors,defence;uranium;technology;metals
avoid_direct_buy_sell_advice,true
```

Spreadsheet warning: some spreadsheet apps may auto-format dates, tickers, or leading zeros. Keep a backup before large manual edits. For safety, use the app validation button after editing CSV by hand.

9. Minimal blank Python file headers

When you first create the blank files, give each one a tiny header so future-you knows what belongs there. These are not finished implementations; they are labels on empty benches.

src/calculations.py

```
"""Portfolio calculation helpers.

Pure functions only where possible. This file should not read files,
write files, call APIs, or use Streamlit directly.
"""

# TODO: calculate current value
# TODO: calculate gain/loss
# TODO: calculate allocation weights
```

src/csv_store.py

```
"""CSV storage helpers for Stock Watchtower.

Responsibilities:
- Load CSV files from data/
- Validate before saving
- Create timestamped backups
- Write safely via temporary files
"""

# TODO: load_holdings_csv
# TODO: save_holdings_csv
# TODO: create_backup
# TODO: safe_write_csv
```

src/validation.py

```
"""Validation helpers for CSV-backed portfolio data."""

# TODO: validate required columns
# TODO: validate numeric fields
# TODO: detect duplicate holding_id values
# TODO: return friendly error messages
```

src/quotes.py

```
"""Quote provider layer.

Start with mock quote data. Later, add real API providers and cache support.
The rest of the app should not care where quote data comes from.
"""

# TODO: get_mock_quotes
# TODO: get_cached_quotes
# TODO: refresh_quotes_from_api
```

app.py

```
"""Stock Watchtower Local CSV Edition.

Run with:
    streamlit run app.py
"""

# TODO: page config
# TODO: sidebar brand area
# TODO: load holdings.csv
# TODO: calculate dashboard values
# TODO: show table and chart
```

10. First useful code scaffold

After the blank canvas is ready, fill only the first milestone. The target is a local dashboard that loads holdings.csv, validates it, applies mock prices, and shows a table plus allocation chart.

requirements.txt

```
streamlit
pandas
plotly
pydantic
python-dotenv
pytest
ruff
```

.gitignore

```
.venv/
__pycache__/*
*.pyc
.env
.DS_Store

# Personal data - keep private unless you intentionally use a private repo
data/*.csv
cache/*.csv
backups/
exports/

# Keep example files if you create them
!data/example_*.csv
```

.streamlit/config.toml

```
[theme]
base = "dark"
primaryColor = "#C7A44B"
backgroundColor = "#101418"
secondaryBackgroundColor = "#1B222A"
textColor = "#E6E0D5"
font = "sans serif"
```

src/theme.py

```
COLORS = {
    "night_charcoal": "#101418",
    "slate_panel": "#1B222A",
    "soft_text": "#E6E0D5",
    "old_gold": "#C7A44B",
    "signal_green": "#4DAA57",
    "risk_red": "#B85656",
    "data_cyan": "#4BA3C7",
}
```

src/mock_data.py

```
import pandas as pd

SAMPLE_HOLDINGS = [
    {
        "holding_id": "h001",
        "name": "Rolls-Royce",
        "ticker": "RR.L",
        "exchange": "LSE",
        "currency": "GBP",
        "asset_type": "stock",
        "quantity": 100,
        "average_cost": 4.25,
        "isin": "GB00B63H8491",
        "figi": "",
        "tags": "defence;uk;growth",
        "notes": "Core UK defence and SMR watch",
    }
```

```

        "created_at": "2026-07-07",
        "updated_at": "2026-07-07",
    },
    {
        "holding_id": "h002",
        "name": "Nokia ADR",
        "ticker": "NOK",
        "exchange": "NYSE",
        "currency": "USD",
        "asset_type": "stock",
        "quantity": 50,
        "average_cost": 4.80,
        "isin": "US6549022043",
        "figi": "",
        "tags": "telecom;adr",
        "notes": "US-listed Nokia ADR",
        "created_at": "2026-07-07",
        "updated_at": "2026-07-07",
    },
]

MOCK_PRICES = {
    "RR.L": 4.76,
    "NOK": 4.62,
}

def sample_holdings_frame() -> pd.DataFrame:
    return pd.DataFrame(SAMPLE_HOLDINGS)

```

src/calculations.py

```

import pandas as pd

def add_portfolio_calculations(holdings: pd.DataFrame, prices: dict[str, float]) -> pd.DataFrame:
    """Return holdings with current value, P/L, and portfolio weight columns."""
    df = holdings.copy()
    df["current_price"] = df["ticker"].map(prices).fillna(df["average_cost"])
    df["quantity"] = pd.to_numeric(df["quantity"], errors="coerce").fillna(0)
    df["average_cost"] = pd.to_numeric(df["average_cost"], errors="coerce").fillna(0)

    df["current_value"] = df["quantity"] * df["current_price"]
    df["total_cost"] = df["quantity"] * df["average_cost"]
    df["gain_loss"] = df["current_value"] - df["total_cost"]

    df["gain_loss_percent"] = 0.0
    non_zero_cost = df["total_cost"] != 0
    df.loc[non_zero_cost, "gain_loss_percent"] = (
        df.loc[non_zero_cost, "gain_loss"] / df.loc[non_zero_cost, "total_cost"] * 100
    )

    total_value = df["current_value"].sum()
    df["portfolio_weight"] = 0.0 if total_value == 0 else df["current_value"] / total_value *
100
    return df

```

src/validation.py

```

import pandas as pd

REQUIRED_HOLDING_COLUMNS = {
    "holding_id", "name", "ticker", "exchange", "currency",
    "asset_type", "quantity", "average_cost"
}

def validate_holdings(df: pd.DataFrame) -> list[str]:
    """Return a list of validation errors. Empty list means valid enough to load."""
    errors: list[str] = []
    missing = REQUIRED_HOLDING_COLUMNS - set(df.columns)
    if missing:
        errors.append(f"Missing required columns: {' '.join(sorted(missing))}")

    if "holding_id" in df.columns and df["holding_id"].duplicated().any():
        errors.append("Duplicate holding_id values found.")

```

```

for col in ["quantity", "average_cost"]:
    if col in df.columns:
        bad = pd.to_numeric(df[col], errors="coerce").isna()
        if bad.any():
            errors.append(f"Column {col} contains non-numeric values.")

return errors

```

src/csv_store.py

```

from pathlib import Path
import pandas as pd
from src.mock_data import sample_holdings_frame

DATA_DIR = Path("data")
HOLDINGS_PATH = DATA_DIR / "holdings.csv"

def ensure_data_files() -> None:
    DATA_DIR.mkdir(exist_ok=True)
    if not HOLDINGS_PATH.exists() or HOLDINGS_PATH.stat().st_size == 0:
        sample_holdings_frame().to_csv(HOLDINGS_PATH, index=False)

def load_holdings() -> pd.DataFrame:
    ensure_data_files()
    return pd.read_csv(HOLDINGS_PATH)

def holdings_csv_bytes() -> bytes:
    ensure_data_files()
    return HOLDINGS_PATH.read_bytes()

```

app.py

```

import streamlit as st
import plotly.express as px

from src.calculations import add_portfolio_calculations
from src.csv_store import load_holdings, holdings_csv_bytes
from src.mock_data import MOCK_PRICES
from src.validation import validate_holdings

st.set_page_config(
    page_title="Stock Watchtower",
    page_icon="assets/icon.png" if False else "■",
    layout="wide",
)

st.sidebar.title("Stock Watchtower")
st.sidebar.caption("Local CSV Edition")
st.sidebar.markdown("---")
st.sidebar.write("Source of truth: `data/holdings.csv`")

st.title("Portfolio Dashboard")
st.caption("Mock prices for now. Real quote providers come later.")

if st.button("Reload CSV"):
    st.cache_data.clear()
    st.rerun()

holdings = load_holdings()
errors = validate_holdings(holdings)

if errors:
    st.error("CSV validation failed.")
    for err in errors:
        st.write(f"- {err}")
    st.stop()

portfolio = add_portfolio_calculations(holdings, MOCK_PRICES)

total_value = portfolio["current_value"].sum()
total_cost = portfolio["total_cost"].sum()
total_gain_loss = total_value - total_cost

a, b, c = st.columns(3)

```

```
a.metric("Total value", f"{total_value:,.2f}")
b.metric("Total cost", f"{total_cost:,.2f}")
c.metric("Unrealised P/L", f"{total_gain_loss:,.2f}")

st.subheader("Holdings")
show_cols = [
    "name", "ticker", "exchange", "currency", "quantity", "average_cost",
    "current_price", "current_value", "gain_loss", "gain_loss_percent", "portfolio_weight"
]
st.dataframe(portfolio[show_cols], use_container_width=True)

st.subheader("Allocation by holding")
fig = px.pie(portfolio, names="ticker", values="current_value", hole=0.45)
st.plotly_chart(fig, use_container_width=True)

st.download_button(
    label="Download holdings CSV",
    data=holdings_csv_bytes(),
    file_name="holdings.csv",
    mime="text/csv",
)
```

11. How to run the first milestone

```
# From inside the project folder
source .venv/bin/activate          # macOS / Linux
# or
.\.venv\Scripts\Activate.ps1      # Windows PowerShell
```

```
streamlit run app.py
```

Streamlit should open a local browser page. If it does not, copy the localhost URL printed in the terminal, usually `http://localhost:8501`, into your browser.

Verification checklist

- The app opens without Python errors.
- The sidebar says Stock Watchtower and Local CSV Edition.
- `data/holdings.csv` exists after the first run.
- The table shows your sample holdings.
- The total value, gain/loss, and chart render.
- The Reload CSV button refreshes the data.
- The Download holdings CSV button downloads a CSV file.

12. Streamlit pages: when to fill them

Do not fill every page immediately. Keep pages as blank canvases until the main dashboard works. Streamlit supports multipage apps through files in the `pages` directory, but a big multipage layout before the core calculations work will slow you down.

Blank page template

```
import streamlit as st

st.set_page_config(page_title="Stock Watchtower - Portfolio", layout="wide")
st.title("Portfolio")
st.info("This page is a placeholder. Build the main dashboard first.")
```

Suggested page build order

Order	Page	What it should do first
1	<code>app.py</code> / Dashboard	Load <code>holdings.csv</code> , calculate values, show table and chart.
2	<code>1_Portfolio.py</code>	Add/edit/delete holdings with CSV backups.
3	<code>2_Transactions.py</code>	Journal buys, sells, dividends, fees, notes.
4	<code>3_News_Research.py</code>	Cache and label news/research items.
5	<code>4_Strategy.py</code>	Store rules and show explainable signals.
6	<code>5_Settings.py</code>	API keys, theme options, paths, export tools.

13. Safe CSV writing and backups

When you add in-app editing, never write directly over `holdings.csv` without a backup. The app should copy the old file into `backups/holdings/` with a timestamp, write the new CSV to a temporary file, then replace the original only after the write succeeds.

```
from datetime import datetime
```

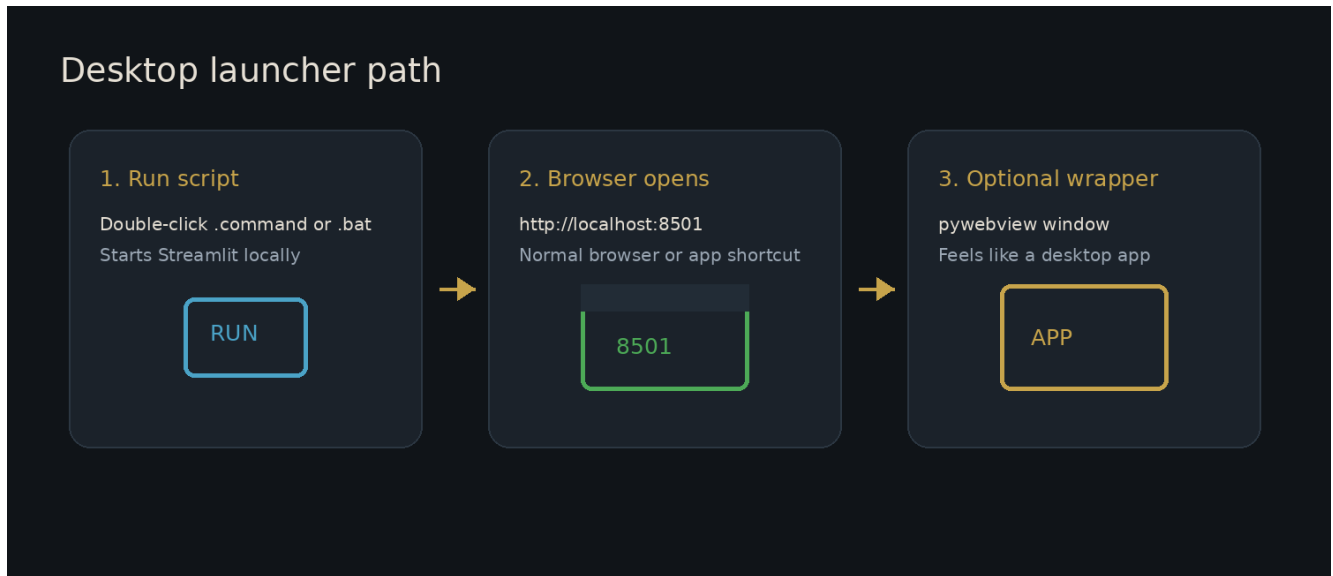
```
from pathlib import Path
import shutil
import pandas as pd

def backup_file(source: Path, backup_dir: Path) -> Path:
    backup_dir.mkdir(parents=True, exist_ok=True)
    stamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    destination = backup_dir / f"{source.stem}_{stamp}{source.suffix}"
    shutil.copy2(source, destination)
    return destination

def safe_write_csv(df: pd.DataFrame, target: Path) -> None:
    temp = target.with_suffix(".tmp")
    df.to_csv(temp, index=False)
    temp.replace(target)
```

This is a small habit that saves projects. A spreadsheet can be recreated; a lost log of your thinking is harder to rebuild.

14. Desktop icon and launcher path



Level 1: simple macOS launcher

```
#!/bin/bash
cd "$(dirname "$0")"
source .venv/bin/activate
python -m streamlit run app.py
```

Save this as `launch_stock_watchtower.command` in the project root, then run `chmod +x launch_stock_watchtower.command`. You can place an alias/shortcut on the desktop. This starts Streamlit and opens your browser.

Level 1: simple Windows launcher

```
@echo off
cd /d "%~dp0"
call .venv\Scripts\activate
python -m streamlit run app.py
pause
```

Save this as `launch_stock_watchtower.bat`. You can right-click it and create a desktop shortcut. Later, change the shortcut icon to `assets/icon.ico`.

Level 2: browser app shortcut

Once Streamlit is running at `localhost:8501`, create a browser shortcut that opens the page. This gives a light desktop feeling without packaging. The drawback is that Streamlit still needs to be running, so the script remains useful.

Level 3: optional pywebview wrapper

When the app is useful, add a native wrapper. `pywebview` can create a desktop window around your local app. This is optional, and it should come after the dashboard and CSV editing are working.

```
import subprocess
import sys
import time
import webview

def main() -> None:
    process = subprocess.Popen(
        [sys.executable, "-m", "streamlit", "run", "app.py", "--server.headless=true"],
    )
    time.sleep(3)
```

```
try:
    webview.create_window("Stock Watchtower", "http://localhost:8501", width=1400,
height=900)
    webview.start()
finally:
    process.terminate()

if __name__ == "__main__":
    main()
```

Do not add pywebview on day one. It introduces desktop-window concerns before the portfolio logic is useful. Treat it as polish, not foundation.

15. Logo, icon, and aesthetic direction

Add design early enough that the app feels like yours, but not so early that visual polish replaces working code. Put all brand decisions in docs/BRAND.md and all images in assets/.

docs/BRAND.md starter

```
# Stock Watchtower Brand Notes

Working name: Stock Watchtower
Alternative names: Raven Ledger, Market Watchtower, Signal Tower

Mood:
- Dark, tactical, analytical
- Ledger-like, calm, serious
- Subtle raven/watchtower symbolism
- A little mythic, not cartoonish

Avoid:
- Crypto-bro neon
- Overloaded trading screens
- Corporate blandness
- Fake luxury finance style

Palette:
- Night charcoal: #101418
- Slate panel: #1B222A
- Soft text: #E6E0D5
- Old gold: #C7A44B
- Signal green: #4DAA57
- Risk red: #B85656
- Data cyan: #4BA3C7

Logo concepts:
1. Watchtower silhouette with a candlestick chart signal beam.
2. Raven head with feather edges shaped like chart bars.
3. Ledger book with a small tower mark stamped in old gold.
4. Circular icon: moon/portfolio pie chart behind a tower.
```

Asset formats

File	Use
assets/logo-mark.png	Sidebar mark, README image, design reference.
assets/logo-horizontal.png	Header/banner version.
assets/icon.png	Streamlit page icon fallback.
assets/icon.ico	Windows shortcut icon.
assets/icon.icns	macOS app icon later.
assets/theme_reference.md	Palette, typography, UI notes.

16. VSCode workflow

Open the whole stock-watchtower folder in VSCode. Do not open individual files as loose tabs only. The integrated terminal should start in the project root, which keeps paths predictable.

Recommended first extensions

- Python extension by Microsoft.
- Pylance for Python language support.
- Ruff for linting/formatting if you want stricter feedback.
- GitLens if you like rich Git history inside VSCode.
- Markdown preview for docs/ files.

Daily loop

1. Pick one task from docs/TASKS.md.
2. Ask ChatGPT for that one task only.
3. Paste or apply code in VSCode.
4. Run `streamlit run app.py`.
5. Fix errors by pasting exact traceback into ChatGPT.
6. When it works, commit to Git.
7. Update PROJECT_CONTEXT.md and TASKS.md.
8. Move to the next task.

17. Git and GitHub

Use GitHub for the code, not necessarily for your private financial data. If the repository is public, do not commit real holdings.csv, transaction logs, notes, API keys, or cache files.

```
git init
git add .
git commit -m "Initial Stock Watchtower scaffold"
```

Good commit messages

```
Add Streamlit dashboard scaffold
Add CSV loading and validation
Add mock price calculations
Add holdings CSV backup workflow
Add desktop launcher scripts
Add brand notes and theme palette
```

18. ChatGPT project prompt

Put this into your ChatGPT Project instructions or paste it at the top of a new coding chat. The aim is to keep the assistant focused on small, runnable increments.

You are my coding partner for Stock Watchtower Local CSV Edition.

Goal:
Build a personal local Python app for portfolio tracking, research notes, and strategy signals.

Stack:

- Python
- Streamlit
- pandas
- Plotly
- CSV files as the database
- VSCode
- GitHub

Rules:

- Build in small, testable milestones.

- Keep code beginner-readable and modular.
- Use CSV files in data/ as the source of truth.
- Create backups before overwriting CSV files.
- External API keys must be stored in .env or Streamlit secrets, never hardcoded.
- Do not add hosting, auth, SQLite, RAG, or real market APIs until asked.
- Strategy outputs must be educational decision-support, not direct buy/sell advice.

When coding:

1. Restate the immediate task briefly.
2. List files to create or modify.
3. Provide complete code for those files.
4. Explain how to run/test it.
5. Mention assumptions and next step.
6. Ask for only the specific existing files you need.

19. First milestone prompt

Create Stock Watchtower Local CSV Edition, Milestone 1.

Goal:

Build a local personal Streamlit app for tracking an investment portfolio using CSV files as the database.

Brand direction:

- Working name: Stock Watchtower
- Mood: dark, tactical, analytical, ledger-like, subtle raven/watchtower symbolism
- Colours: charcoal, slate, off-white, old gold, muted green, muted red
- Avoid: crypto-bro neon, clutter, cartoonish finance graphics

Requirements:

- App runs locally with streamlit run app.py.
- Use data/holdings.csv as the source of truth.
- If holdings.csv does not exist, create it from sample holdings.
- Load holdings from CSV.
- Validate required columns.
- Show a portfolio dashboard table.
- Use mock current prices for now.
- Calculate current value, gain/loss, gain/loss percentage and portfolio weight.
- Show total portfolio value.
- Show one allocation chart by holding.
- Add a Reload CSV button.
- Add a Download holdings CSV button.
- Create docs/BRAND.md with initial aesthetic notes.
- Do not add real market APIs yet.
- Do not add SQLite.
- Do not add authentication.
- Do not add hosting.
- Do not add pywebview yet.

Output:

1. Folder structure.
2. Complete code for each file.
3. Commands to run in VSCode terminal.
4. How to verify it works.
5. Next recommended milestone.

20. Build roadmap

Milestone	Build	Definition of done
0	Scaffold and blank files	Folders exist, blank files named clearly, venv works.
1	Mock dashboard	CSV loads, calculations work, table/chart render.
2	In-app holdings editing	Add/edit/delete holdings, validate, backup before save.
3	Desktop launcher	.command and .bat launch Streamlit locally.
4	Brand pass	Logo/icon placeholder, Streamlit theme, sidebar identity.
5	Transactions	transactions.csv, add/view/filter/export records.
6	Quote provider layer	Mock provider swapped for real provider with cache.
7	News/research	news_cache.csv, source/date/link/notes.
8	Strategy rules	Rules produce explainable watch signals, not buy/sell commands.
9	pywebview wrapper	Optional native window around localhost app.
10	Upgrade decision	Stay CSV, move to SQLite, or split into FastAPI/frontend.

21. Troubleshooting

Streamlit command not found

```
# Use the Python module form instead
python -m streamlit run app.py
```

This usually means the virtual environment is not activated, or Streamlit is not installed in the active environment.

ImportError: No module named src

Make sure you run Streamlit from the project root, not from inside src/. The command should be run while your terminal path is stock-watchtower/.

CSV validation failed

Open data/holdings.csv and check the header row. The required columns are holding_id, name, ticker, exchange, currency, asset_type, quantity, and average_cost. Spelling matters.

Numbers look wrong after spreadsheet editing

Check whether your spreadsheet app changed decimals, dates, or tickers. Save as plain CSV, not XLSX. If in doubt, restore the latest backup from backups/holdings/.

Git wants to commit data files

Check .gitignore. If data files were already tracked, .gitignore will not remove them automatically. You can untrack them with `git rm --cached data/holdings.csv`, then commit the change.

22. Upgrade decisions

Stay with CSV until it gets in your way. Upgrade only when there is a concrete pain point.

Pain point	Upgrade
Manual CSV edits cause too many mistakes	Use in-app editing with stronger validation first.

CSV files become slow or awkward	Move to SQLite.
You want phone/cloud access	Deploy Streamlit or use a remote machine/Tailscale style setup.
You want real API endpoints	Split into FastAPI backend and frontend later.
You want scheduled alerts/RSS	Add a local scheduler or small backend worker.
You want heavy RAG	Create a separate research module with its own storage.

The right order is simple: build the useful cockpit first, then bolt on engines. Do not start by building the airport.

23. Appendix: complete initial README.md

Stock Watchtower Local CSV Edition

A personal local portfolio dashboard built with Python, Streamlit, pandas, Plotly, and CSV files.

Current status

Milestone 1: local mock dashboard.

Run locally

```
```bash
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
streamlit run app.py
```
```

On Windows PowerShell:

```
```powershell
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
streamlit run app.py
```
```

Data

The source of truth is `data/holdings.csv`.

Do not commit real personal financial data to a public GitHub repo.

Roadmap

- Mock dashboard
- CSV validation
- In-app holdings editing
- CSV backups
- Desktop launcher
- Transactions journal
- Quote provider and cache
- News/research page
- Strategy signals

24. Appendix: docs/TASKS.md starter

Stock Watchtower Tasks

Now

- [] Create folder structure and blank files
- [] Create Python virtual environment
- [] Add requirements.txt
- [] Add first holdings.csv
- [] Build mock Streamlit dashboard

Next

- [] Add CSV validation errors in UI
- [] Add safe CSV backups
- [] Add in-app add/edit/delete holdings
- [] Add desktop launcher scripts
- [] Add icon/logo placeholder

Later

- [] Add transactions.csv and transaction page
- [] Add quote provider layer
- [] Add real quote API and cache
- [] Add news/research cache
- [] Add strategy profile and signals
- [] Add pywebview wrapper

25. Appendix: docs/PROJECT_CONTEXT.md starter

Stock Watchtower Project Context

Goal

Build a personal local portfolio dashboard for tracking holdings, transactions, research notes, and strategy signals.

Current architecture

- Python
- Streamlit
- CSV files as local storage
- No hosting
- No auth
- No database yet

Source of truth

- data/holdings.csv for holdings
- data/transactions.csv later for transaction journal
- data/strategy_profile.csv later for strategy rules

Design direction

Dark, tactical, analytical, ledger-like, subtle watchtower/raven symbolism.

Current milestone

Milestone 1: load holdings.csv, use mock prices, calculate portfolio values, show table and chart.

Guardrails

- Do not expose API keys.
- Do not commit private financial data to a public repo.
- Build in small testable steps.
- Strategy outputs are educational decision-support, not direct buy/sell advice.

26. References

These are the official or primary references used for the technical direction in this guide.

Streamlit - Run your app: <https://docs.streamlit.io/develop/concepts/architecture/run-your-app>

Streamlit CLI run reference: <https://docs.streamlit.io/develop/api-reference/cli/run>

Streamlit theming overview: <https://docs.streamlit.io/develop/concepts/configuration/theming>

Streamlit config.toml reference: <https://docs.streamlit.io/develop/api-reference/configuration/config.toml>

Python venv documentation: <https://docs.python.org/3/library/venv.html>

pywebview usage guide: <https://pywebview.flowrl.com/3.7/guide/usage.html>

pywebview API guide: <https://pywebview.flowrl.com/3.7/guide/api.html>

Note: APIs, hosting plans, and tool behaviour can change. For this local-first build, the most important dependencies are Streamlit, Python, pandas, and your CSV files. Keep the app simple until the basic workflow is dependable.